TP02 — Programación I

Controlador de misiones: funciones, arreglos y decisiones

Robot: Unitree Go2 (simulado con MuJoCo)
Grupo: 3
Comisión: .....
Materia: Programación I — Carreras de Informática, UADE

Qué es este repositorio

Este proyecto implementa un controlador de misiones para el robot Unitree Go2, capaz de ejecutar cualquier secuencia de comandos representada como una lista, en lugar de una secuencia fija de instrucciones. El desarrollo siguió un diseño Top-Down: primero se definieron las funciones necesarias y sus responsabilidades, y luego se implementó cada una por separado.

El programa corre contra el gemelo digital (simulador MuJoCo) provisto por la cátedra, no contra el robot físico.

Qué hace el programa

- Recibe una misión: una lista de tuplas, donde cada tupla indica una acción (avanzar, girar, detenerse, saludar) y sus parámetros (velocidad y tiempo).
- Valida el formato de cada comando antes de ejecutarlo: que sea una tupla o lista, que el tipo exista, que tenga la cantidad correcta de parámetros, que velocidad y tiempo sean números y que el tiempo sea positivo.
- Ejecuta los comandos válidos sobre el robot simulado.
- Si un comando tiene formato correcto pero excede los límites físicos del simulador (0.2 m/s, 0.5 rad/s, 10 s por orden), el robot lo rechaza con ErrorDeSeguridad y el programa lo registra sin detenerse.
- Al finalizar cada misión, genera un reporte con comandos totales, ejecutados y rechazados, distancia recorrida, duración estimada y el motivo de cada rechazo.

Se probó con cuatro misiones (MISION_BASICA, MISION_CON_ERRORES, MISION_CUADRADO y una misión propia) sin modificar ninguna función entre una y otra.

Por restricción de la cátedra, el código no usa variables globales: todo está dentro de funciones, salvo los import.

Estructura de la entrega

```
TP02_Grupo3/
|-- README.pdf            -> este archivo
|-- BITACORA.pdf          -> registro cronológico del proceso
|-- PROBLEMAS.pdf         -> fichas de los problemas encontrados y cómo se resolvieron
|-- CONCLUSIONES.pdf      -> reflexión final y respuestas de cierre
|-- ESQUEMA_TOPDOWN.pdf   -> esquema de funciones
|-- INFORME_BREVE.pdf     -> informe de una página
|-- codigo/
|   `-- tp02_frutos.py   -> el controlador de misiones
`-- evidencias/
    |-- 01_entorno.pdf          -> caracterización inicial e instalación de Python y Git
    |-- 02_instalacion.pdf      -> instalación de MuJoCo, Visual C++ y estructura del proyecto
    |-- 03_errores.pdf          -> capturas de cada problema documentado
    |-- 04_soluciones.pdf       -> capturas del estado resuelto de cada problema
    |-- 05_gemelo_digital.pdf   -> puesta en marcha del simulador con el Go2
    `-- 06_misiones.pdf         -> ejecución de las cuatro misiones con sus reportes
```

Entorno utilizado

Componente	Versión
Sistema operativo	Windows 10.0.26200.8655 (AMD64)
Python	3.14.7
MuJoCo	3.13.0
Git	2.55.0.windows.5
Robot	Unitree Go2 (cuadrúpedo), modo simulado

Cómo ejecutar el programa

1. Clonar el repositorio del laboratorio: git clone https://github.com/tsamaan/UadeRobotLab.git
2. Instalar las dependencias según INSTALACION.md (Python 3.10+, MuJoCo vía pip).
3. Copiar codigo/tp02_frutos.py como mi_desarrollo/mi_tp02.py dentro de TP02_Programacion_I.
4. Levantar el simulador desde la carpeta TP02_Programacion_I con INICIAR_SIMULADOR.bat y elegir la opción 2 - Go2.
5. Con el simulador abierto, en otra terminal, ejecutar EJECUTAR_MI_CODIGO.bat.

Funciones

Función	Responsabilidad
es_numero(valor)	Devuelve True si el valor es int o float (usa type(), así un bool no pasa como número)
comando_es_valido(comando)	Valida el formato del comando, sin tocar el robot
ejecutar_comando(robot, comando)	Ejecuta un comando ya validado y atrapa el rechazo del robot si excede sus límites
ejecutar_mision(robot, mision, historial)	Recorre la misión, usa las dos funciones anteriores y registra cada resultado en el historial
generar_reporte(historial)	Calcula y muestra las estadísticas finales
correr_mision(robot, nombre, mision)	Crea un historial nuevo, ejecuta la misión, genera el reporte y detiene el robot
crear_mi_mision()	Devuelve la lista de comandos de la misión propia
main()	Conecta el robot, corre las cuatro misiones y desconecta